Étude de la confiance accordée à un humain vs une IA

[À MODIFIER: Prénom NOM], [À MODIFIER: Prénom NOM]

Master 1 Génie Logiciel
Faculté des Sciences – Département Informatique

28 mai 2026

Plan

1. Introduction
2. Contexte du Projet
3. Développement
   - Architecture générale
   - Plateforme Web
   - Application Python
   - Intégration Web $\leftrightarrow$ Python
4. Protocole Expérimental
5. Résultats Attendus
6. Conclusion

Motivation

- **Question fondamentale** : Les utilisateurs accordent-ils différemment leur confiance à un adversaire humain vs une IA ?
- **Contexte** : Omniprésence de l'IA dans les interactions quotidiennes
- **Enjeu** : Conception d'interfaces homme-machine éthiques et fiables

État de l'art

Théorie de l'attribution

- Heider (1958), Weiner (1985)
- Explications causales biaisées pour les machines

Étude Ullman et al. (2014)

- Yale University
- Confiance vs tromperie du robot

Anthropomorphisme inversé

- Standards moraux différents selon la cible
- Machines jugées plus sévèrement

Hypothèses de recherche

H_1 Les participants expriment une évaluation de déloyauté **significativement plus élevée** pour le CheatBot que pour le même comportement attribué à un humain

H_{2a} L'écart de confiance **diminue** au fil des tours (apprentissage)

H_{2b} L'ordre de présentation **influence** les perceptions ultérieures

De Pepper à la plateforme digitale

Phase 1 : Robot Pepper

- Objectif initial : humain vs humanoïde vs digital
- Intégration avec robot social SoftBank

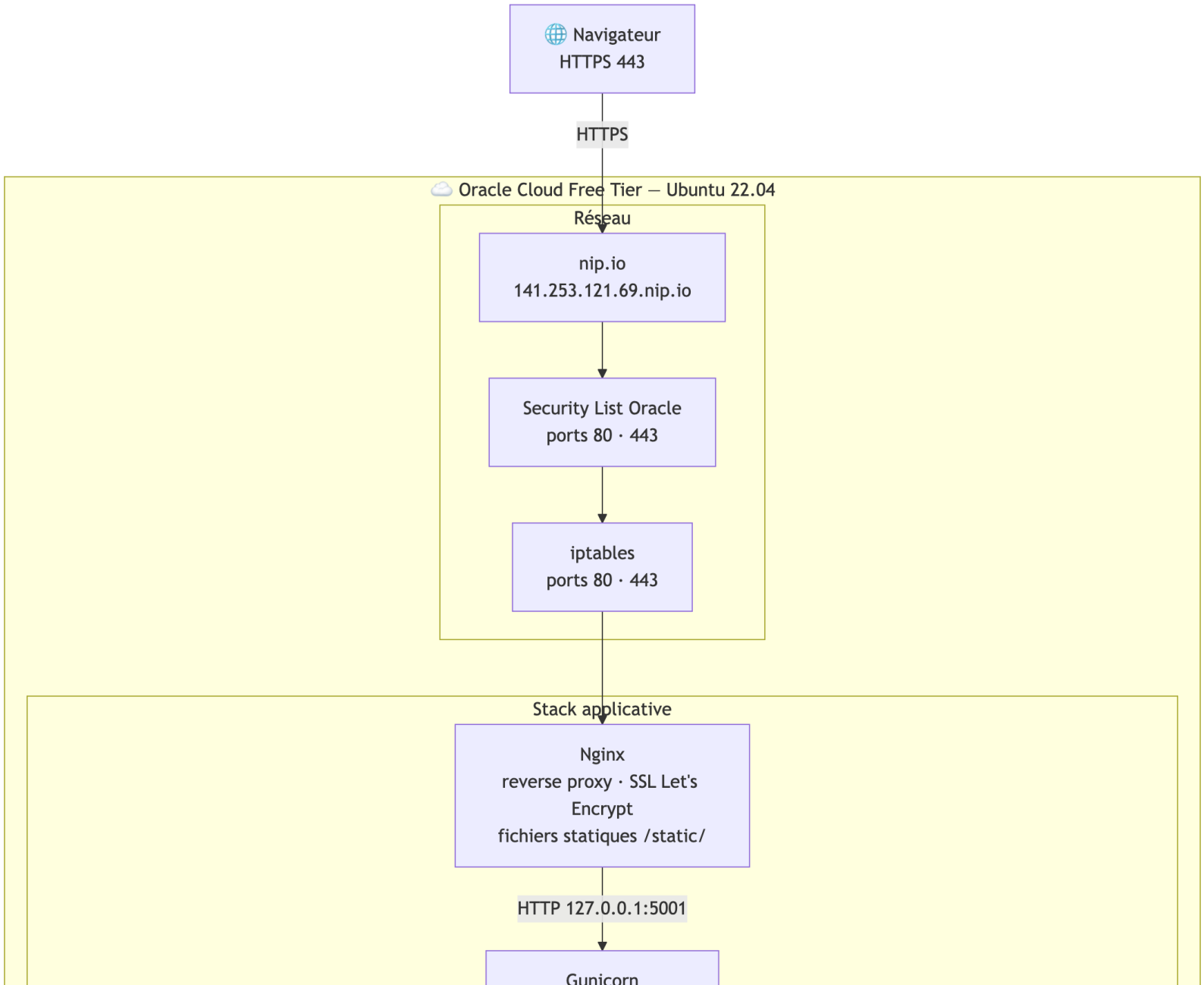
Problèmes rencontrés

- Parties trop longues (15-30s latence)
- Confonds expérimentaux
- Expérience frustrante

Phase 2 : Pivot vers digital

- Maintien de Python (infrastructure Pepper déjà développée)
- Ajout de couche web (Flask)
- Avantage : contrôle rigoureux, reproductibilité, scalabilité

Architecture générale



Stack technologique

Backend Python

- Python 3.x
- Architecture modulaire
- SQLite
- Socket TCP/IP

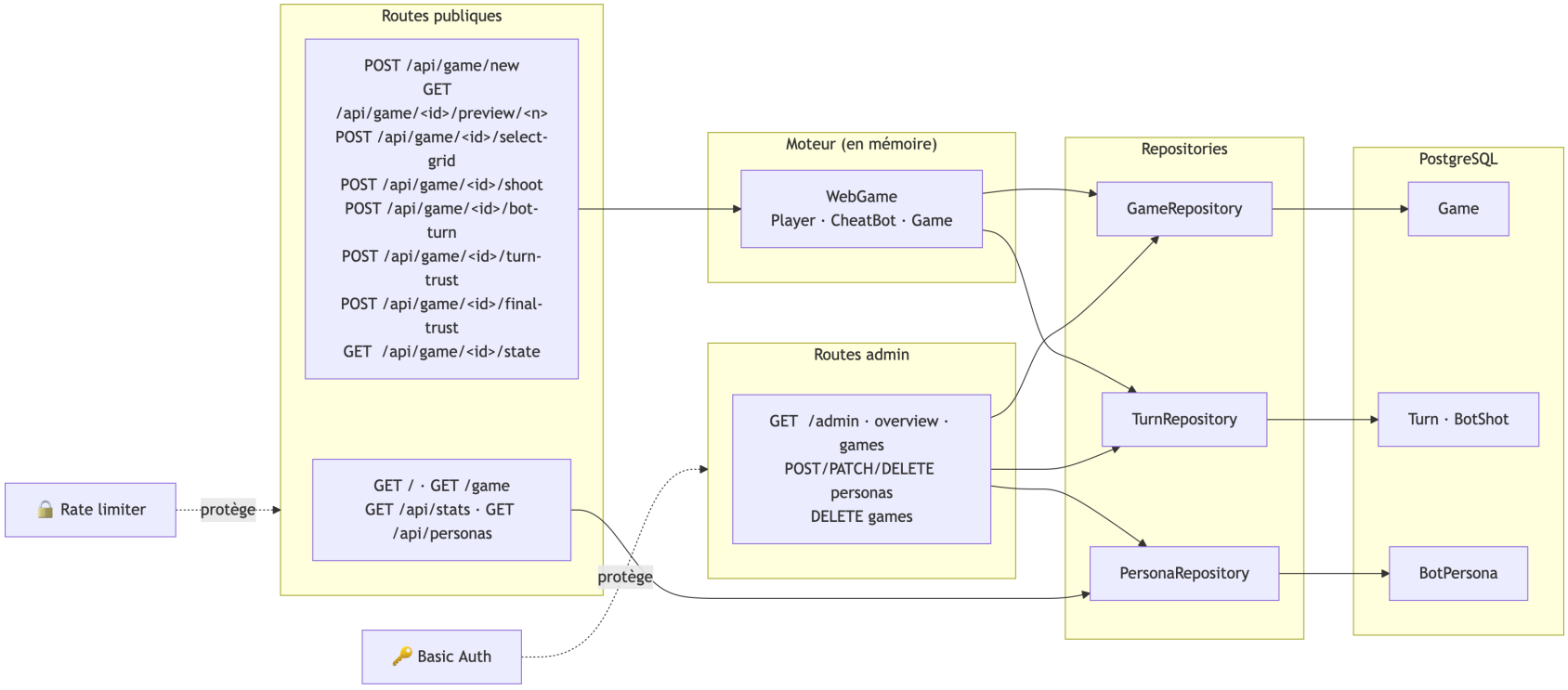
Frontend Web

- Flask 2.3+
- API REST
- HTML5 + CSS3
- JavaScript vanilla

Communication

- API REST JSON
- Session management
- Base SQLite centralisée

Plateforme Web : Vue d'ensemble



Landing Page (index.html)

Sections principales :

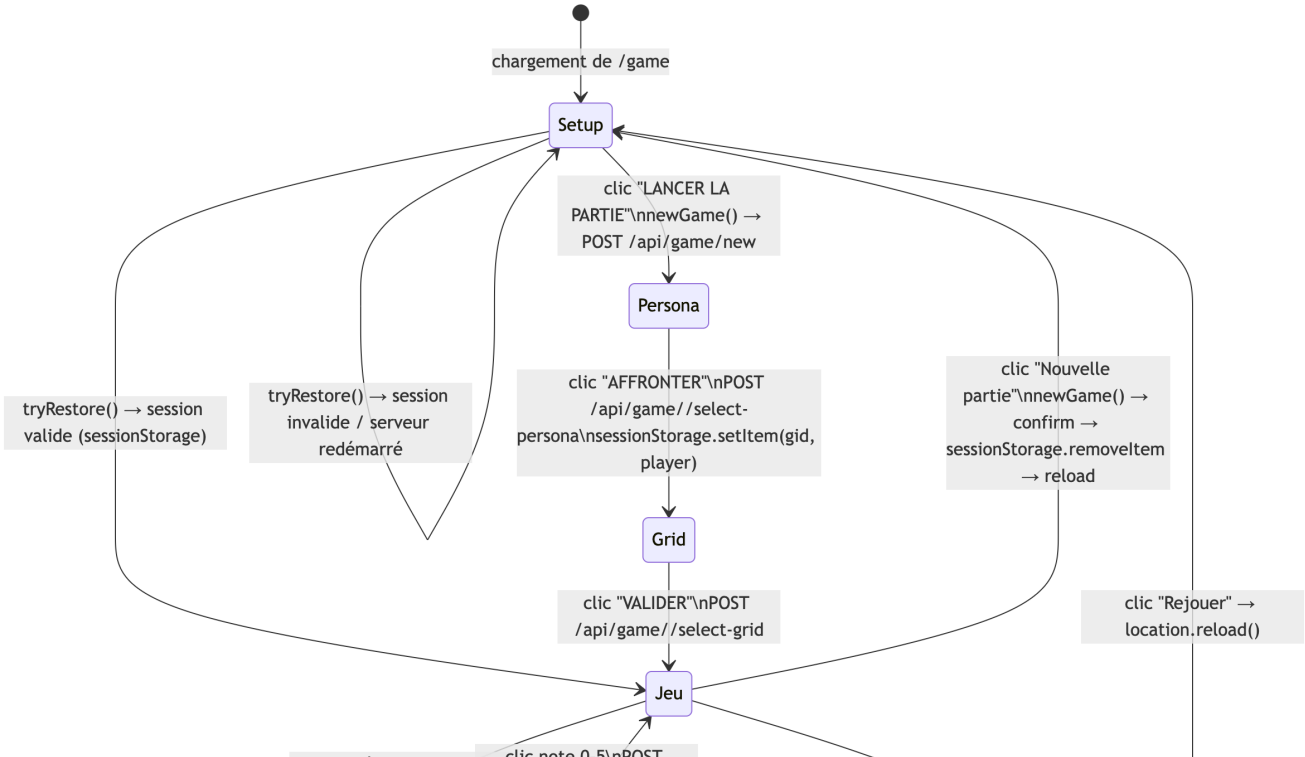
1. Navigation
2. Héros (appel à l'action)
3. 6 cartes de fonctionnalités
4. Tutoriel (4 étapes)
5. Section recherche



Interface de Jeu Web

Machine à états (5 phases) :

1. **Setup** : Entrée du nom du joueur
2. **Persona** : Sélection du contexte (humain vs robot)
3. **Grid** : Validation configuration de flotte
4. **Playing** : Jeu et collecte de scores de confiance
5. **GameOver** : Résultats et débriefing



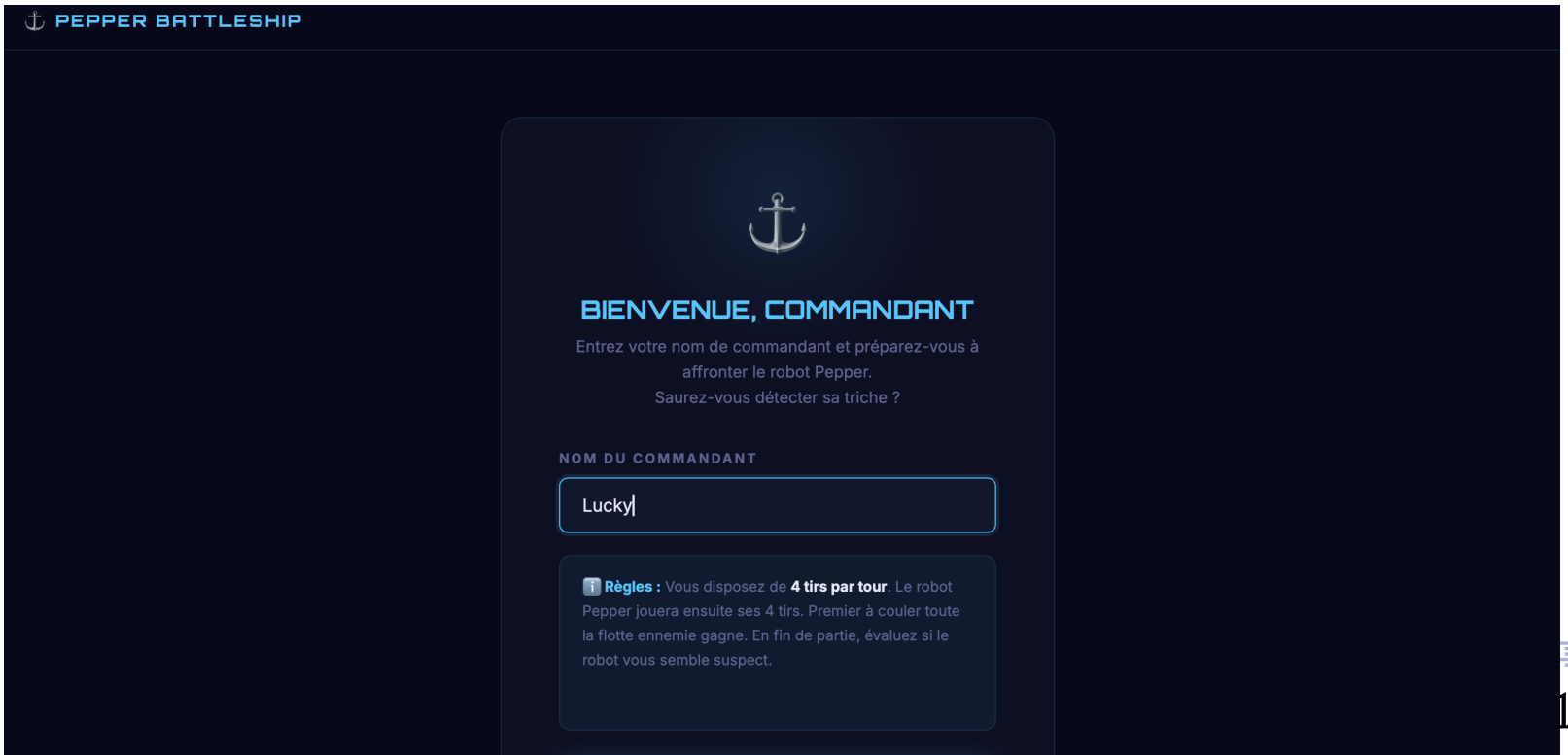
Grilles de jeu (10×10)

Grille du joueur :

- Ma flotte (6 bateaux)
- Tirs ennemis reçus
- Symboles : X C *

Grille de suivi :

- Mes tirs effectués
- Résultats (touché/raté/coulé)
- Visualisation en temps réel

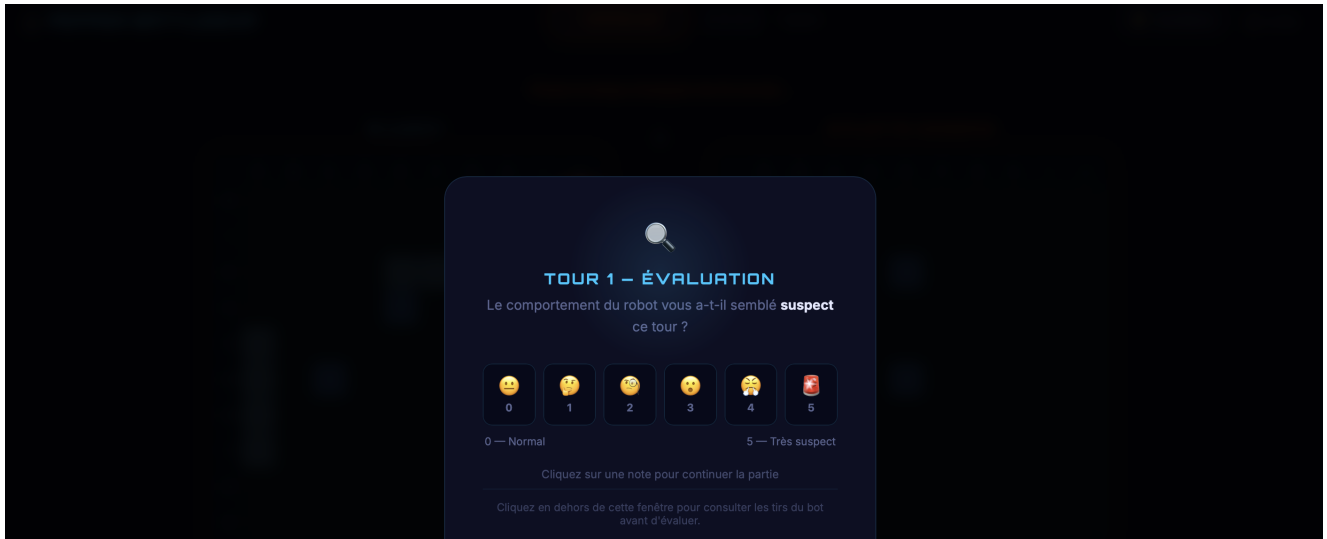


Système de notation de confiance

Après chaque tour du bot :

"Dans quelle mesure pensez-vous que le bot a triché ?"

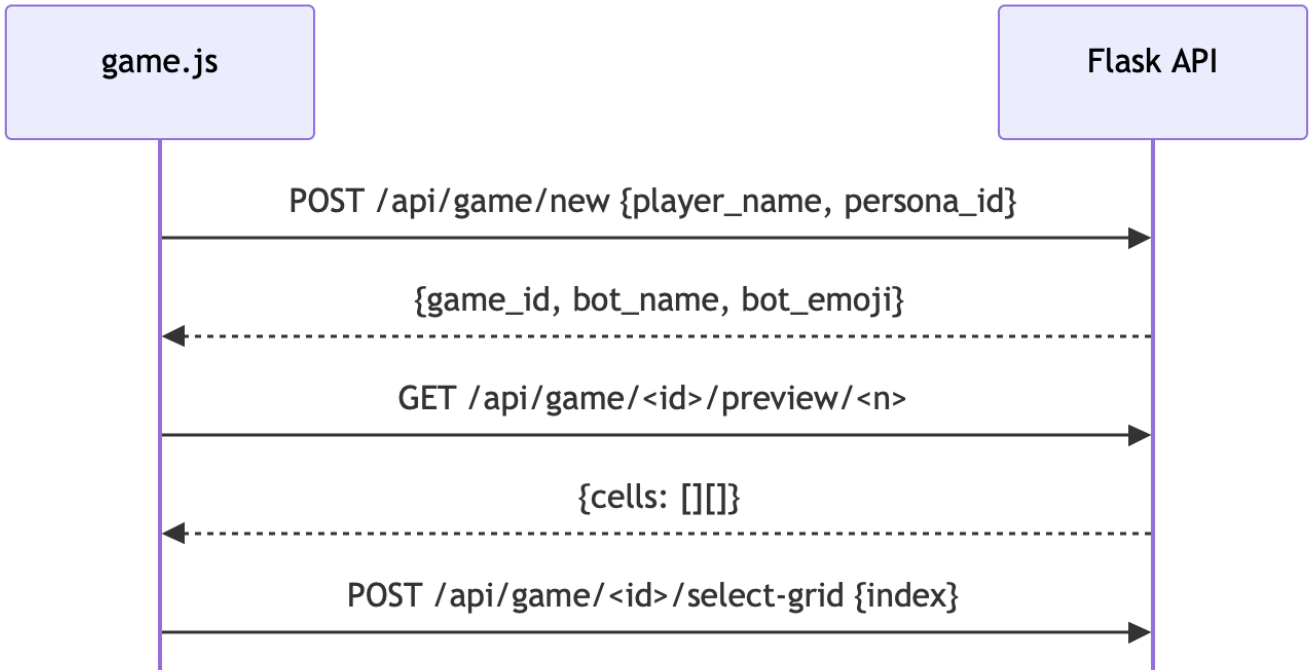
Score	Interprétation
0	Pas du tout triché
1-2	Peu probable
3	Incertain
4-5	Très suspect / Certainement triché



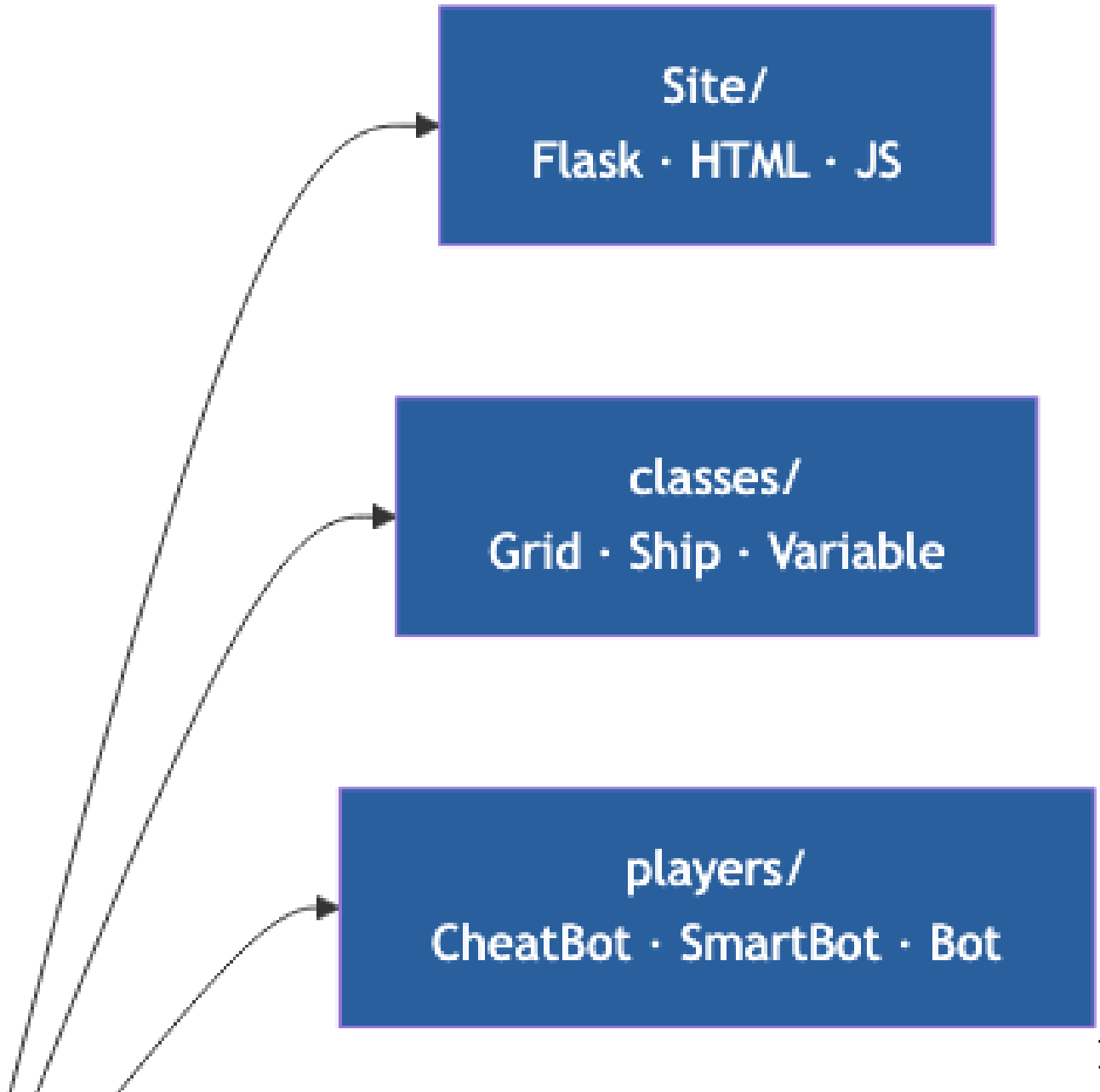
API REST

Routes principales :

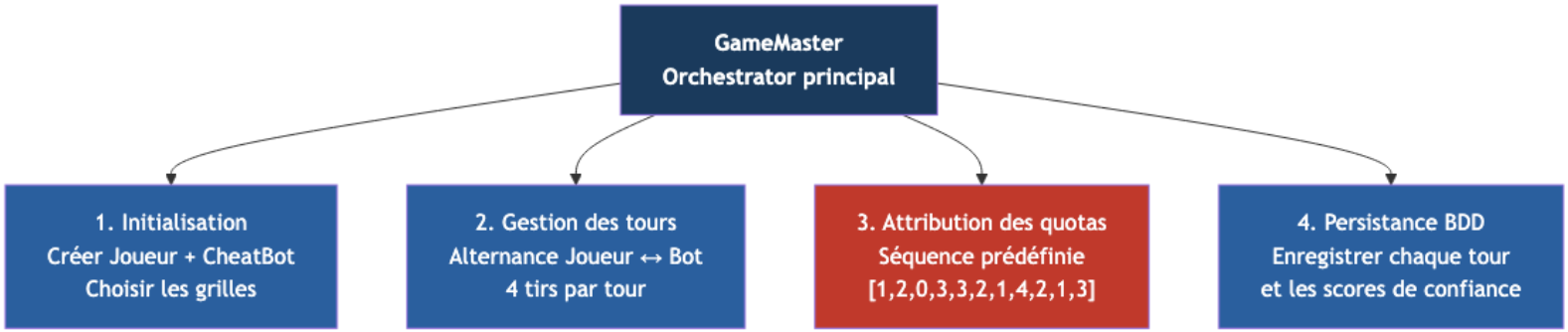
- `GET /` : Landing page
- `POST /api/game/new` : Créer partie
- `POST /api/game/{id}/shoot` : Tir joueur
- `POST /api/game/{id}/bot-turn` : Tour du bot
- `POST /api/game/{id}/turn-trust` : Enregistrer score
- `GET /api/stats` : Statistiques globales



Architecture modulaire Python



GameMaster (Orchestrateur principal)



4 responsabilités :

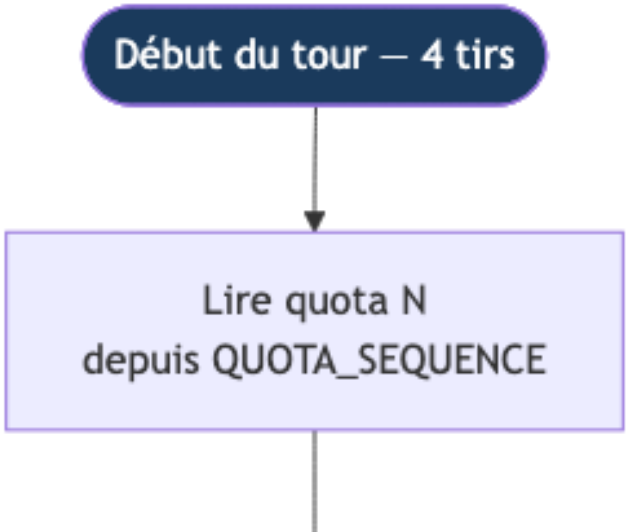
1. Initialisation des joueurs (humain vs CheatBot)
2. Gestion des tours de jeu
3. Attribution des quotas de triche
4. Enregistrement en base de données

CheatBot : Architecture de triche

Le cœur du protocole expérimental

Processus par tour :

1. Accès à la grille ennemie (si mode digital)
2. Identification des cases de bateaux non touchées
3. Sélection de N cases bateaux (succès)
4. Sélection de $4 - N$ cases vides (échecs)
5. Mélange aléatoire (éviter patterns)
6. Exécution des 4 tirs

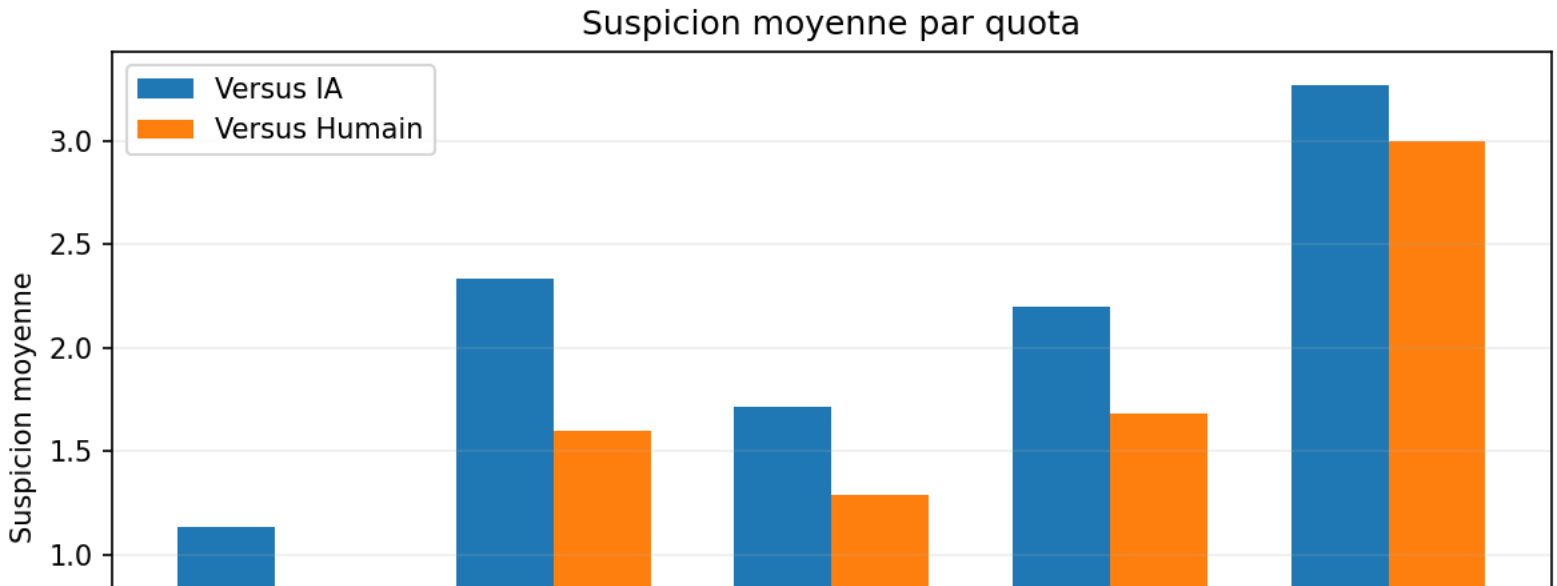


CheatBot : Quotas et équivalence

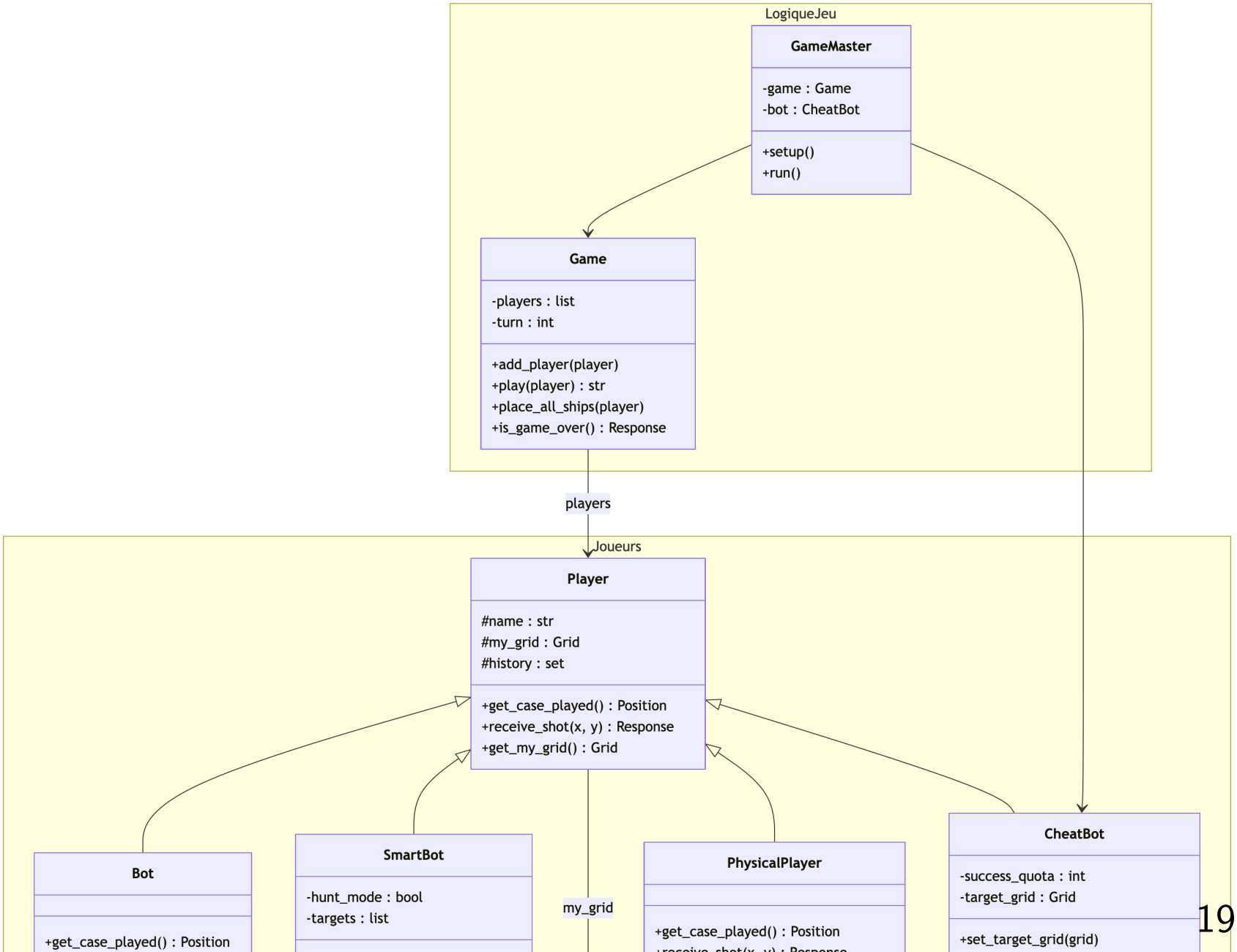
Garantie de contrôle expérimental :

- Quota prédéfini par tour (ex : [2, 1, 3, 2, 1, ...])
- **Même séquence** utilisée dans les deux conditions
- Adaptation automatique si moins de bateaux disponibles
- Communication socket optionnelle vers Pepper

Résultat : Comportement observable **strictement identique** dans les deux parties, seul le label varie.



Hiérarchie des joueurs



Modèles de données

Grid (10×10)

- Navires placés
- État de chaque case
- Méthode `shoot(x,y)`

Ship

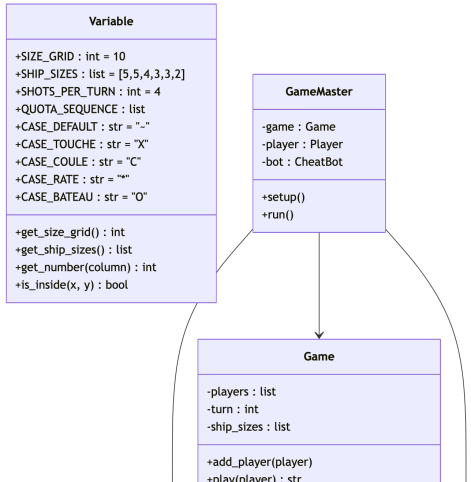
- Position, orientation
- Taille (5, 4, 3, 3, 2)
- Détection coulage

Position

- Coordonnées (x, y)
- Validation limites

Response

- Message (touché/coulé/raté)
- État après tir



Base de données SQLite

Schéma relationnel :

Table Game

- game_id (PK)
- player_name
- player_type
- date_played
- winner

Table Turn

- turn_id (PK)
- game_id (FK)
- turn_number
- bot_quota
- trust_score

Table BotShot

- shot_id (PK)
- turn_id (FK)
- shot_number
- pos_x, pos_y
- result

Game			
SERIAL	id	PK	
TEXT	player_name		
TEXT	player_type		Digital-Web
TIMESTAMP	date_played		DEFAULT CURRENT_TIMESTAMP
TEXT	winner		nom joueur ou bot gagnant
INTEGER	trust_score		score moyen de confiance
INTEGER	trust_final		1 = détecté · 0 = non détecté

contient

BotPersona			
SERIAL	id	PK	
TEXT	name		UNIQUE
TEXT	emoji		DEFAULT 🤖
TEXT	description		
BOOLEAN	active		DEFAULT TRUE

Interfaces utilisateur

Mode Console

- Affichage textuel
- Interaction CLI
- Tests et développement

Mode Graphique (Tkinter)

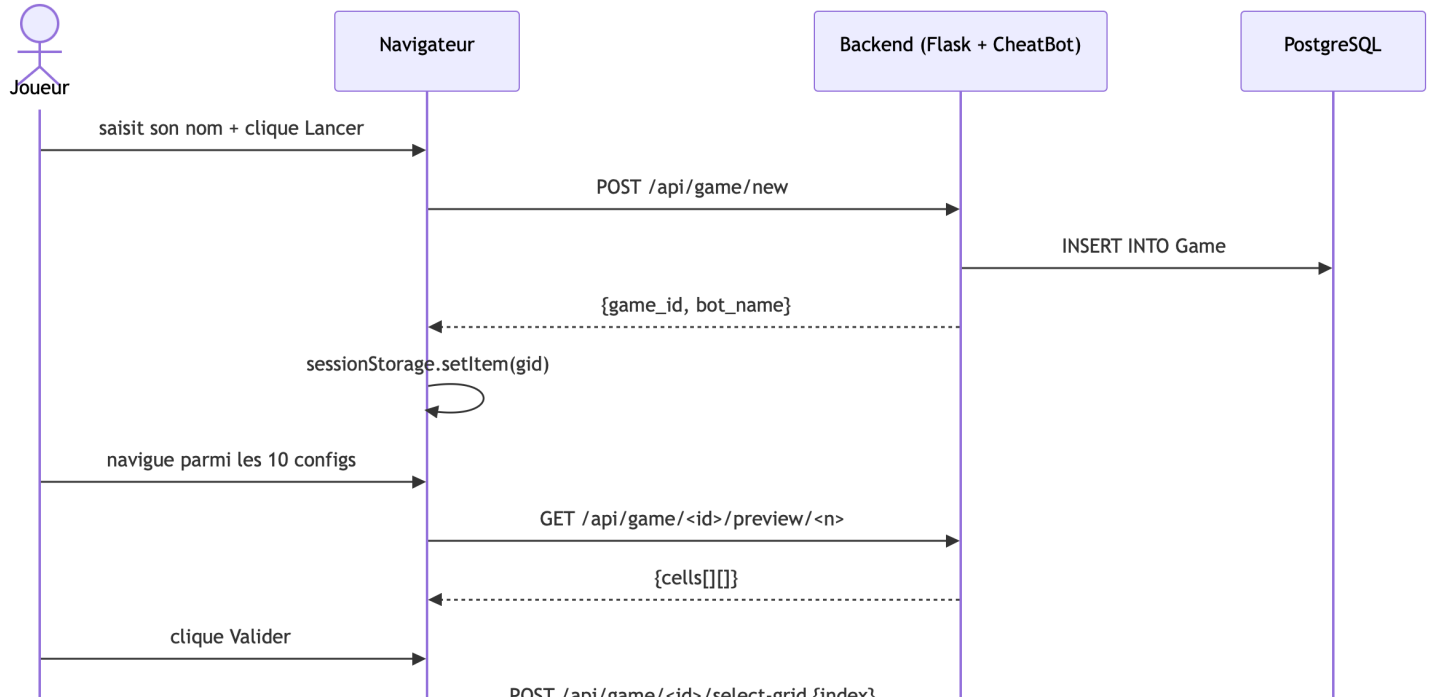
- Canvases 10×10
- Visualisation intuitive
- Intégration questionnaire

Image / Schéma : Capture d'écran : Interface Tkinter

Synchronisation Web ↔ Python

Flux de données complet :

1. Utilisateur accède à `/game` (Flask)
2. Flask initialise GameMaster en Python
3. Tirs joueur → API `/shoot` → logique Python
4. Tour du bot → appel CheatBot.get_case_played()
5. Résultats → enregistrement SQLite
6. Scores de confiance → base de données



Avantages de cette architecture

- **Interface moderne** : Web pour UX optimale et recrutement en ligne
- **Logique robuste** : Python éprouvé et testé
- **Contrôle rigoureux** : Comportement identique dans les deux conditions
- **Reproductibilité** : Code centralisé et versionné
- **Scalabilité** : Collecte de données auprès de nombreux participants
- **Flexibilité future** : Intégration possible avec Pepper (socket TCP/IP)

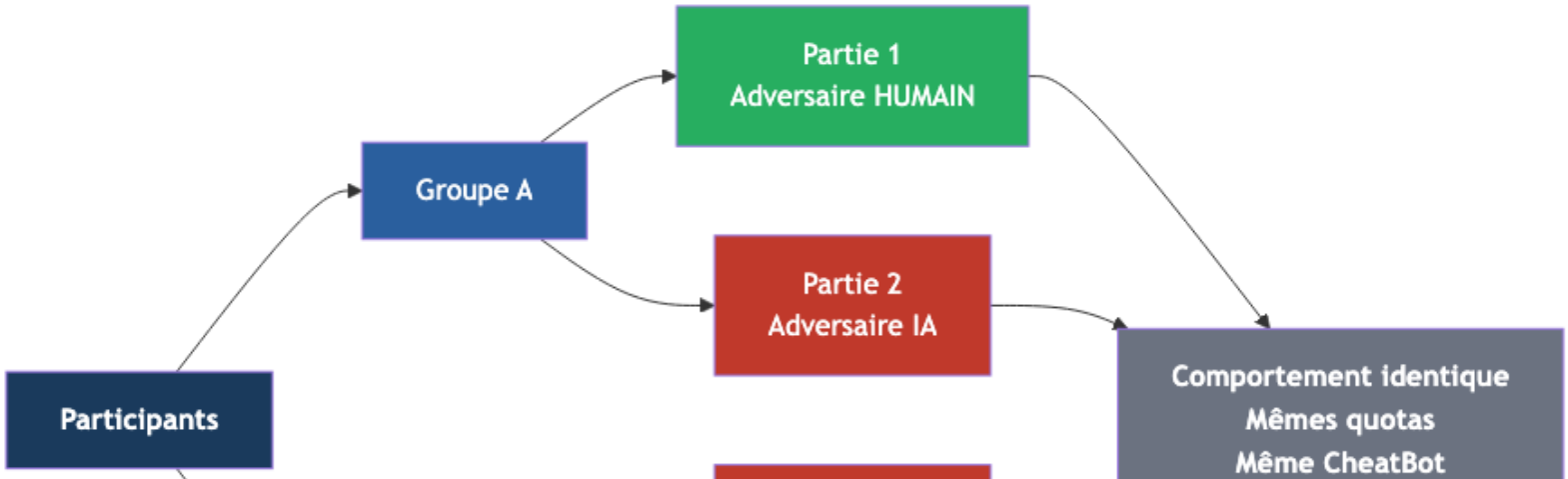
Design quasi-expérimental

Plan intra-groupe (within-subjects) :

Chaque participant joue **2 parties** avec comportement adversaire **identique**

	Partie 1	**Partie 2**
Condition A	Adversaire X (Label 1)	Adversaire Y (Label 2)
Condition B	Adversaire Y (Label 2)	Adversaire X (Label 1)

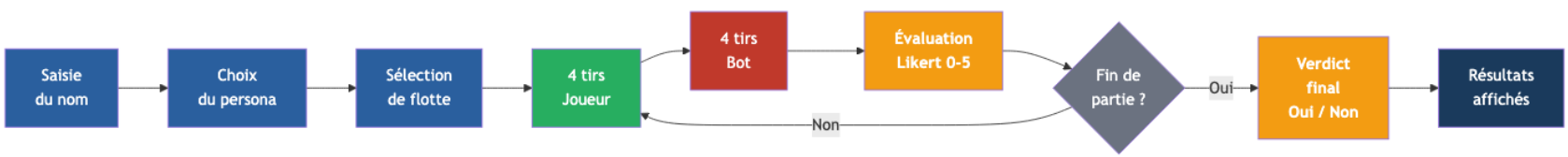
Label 1 = Humain (adversaire présenté comme joueur humain)
Label 2 = IA (adversaire présenté comme CheatBot)



Déroulement des parties

Phases d'une partie :

1. Sélection du nom du joueur
2. **[À MODIFIER: [Questionnaire démographique - À préciser]]**
3. Sélection de la configuration de flotte (10 grilles prédéfinies)
4. Combat (alternance joueur $\leftrightarrow$ bot, 4 tirs/tour)
5. Évaluation de confiance après chaque tour du bot (0-5)
6. Fin : Victoire ou défaite
7. **[À MODIFIER: [Questionnaire post-expérience - À développer]]**



Mesure de confiance

Variable dépendante principale :

À la fin de chaque tour du bot, le joueur répond :

"Dans quelle mesure pensez-vous que votre adversaire a triché ?"

Échelle de Likert 0-5 :

- 0 = Pas du tout triché
- 1-2 = Peu probable
- 3 = Incertain
- 4-5 = Très suspect / Certainement triché

Enregistrement : Horodaté et stocké immédiatement en base

Variables contrôlées

Garantie d'équivalence comportementale :

- **Même adversaire** : Code CheatBot identique
- **Même quota** : Séquence [q1, q2, ..., qN] répétée
- **Même grilles** : Configurations prédéfinies identiques
- **Même ordre** : Séquence de quotas appliquée dans l'ordre

Variable indépendante manipulée :

- Label de l'adversaire (Humain vs CheatBot)



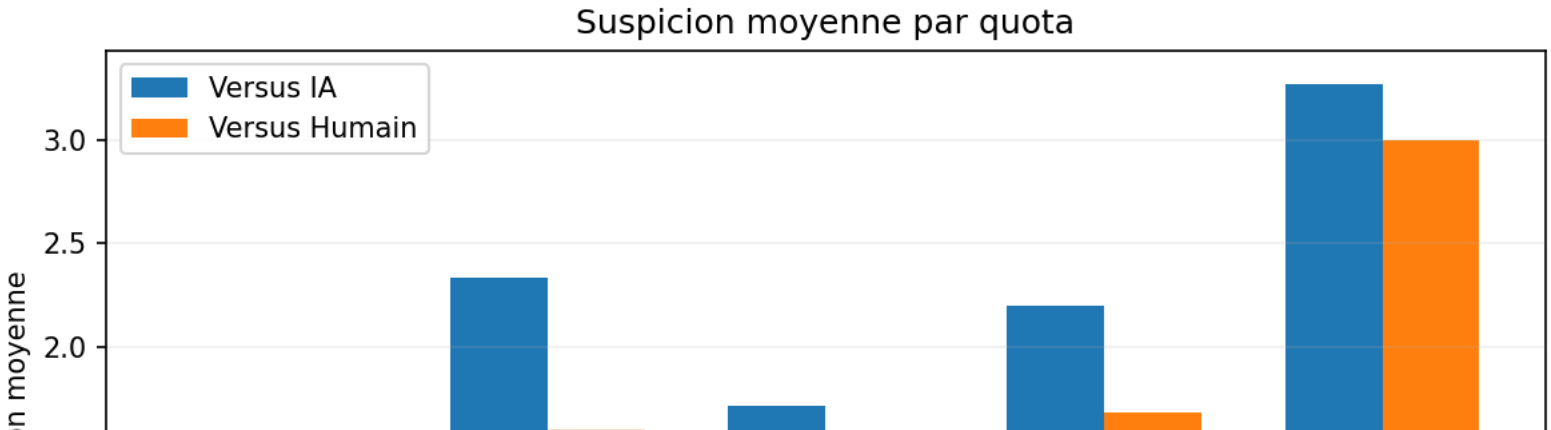
Analyse prévue

Analyses principales :

1. Test de comparaison des moyennes (scores de confiance par condition)
2. Calcul de la magnitude de l'effet (Cohen's d)
3. Analyse temporelle (évolution des scores au fil des tours)
4. **[À MODIFIER: [Analyses statistiques supplémentaires à définir]]**

Hypothèses à tester :

- H_1 : Différence significative entre les deux labels
- H_{2a} : Diminution de l'écart au fil du temps
- H_{2b} : Effet d'ordre de présentation



Implications attendues

Si H_1 est confirmée :

- Les utilisateurs appliquent des standards différents selon le label
- Biais systématique dans la perception de tromperie artificielle
- Implications pour le design éthique d'IA

Si H_1 est réfutée :

- La perception de déloyauté est basée sur le comportement, pas le label
- Questions sur la nature du biais anthropomorphe

Résultats secondaires :

- Profils individuels de sensibilité à la triche
- Rôle de l'expérience préalable avec l'IA
- **[À MODIFIER: [Analyses explorathoires à définir]]**

Contribution et avancées

Apports scientifiques :

- **Théorique** : Éclairage sur les mécanismes de confiance asymétrique
- **Méthodologique** : Protocole reproductible et rigoureux
- **Applicatif** : Recommandations pour design d'IA responsable

Particularités du projet :

- Architecture web+Python flexible et scalable
- Contrôle expérimental absolu du comportement adversaire
- Collecte de données centralisée
- Intégrabilité future avec robot physique

Perspectives futures

- **Extension** : Réintégration du robot Pepper (bottleneck résolu)
- **Variations** : Différents contextes compétitifs (poker, jeux de données)
- **Longitudinal** : Suivi de la confiance chez mêmes participants
- **Démographique** : Étude des facteurs individuels modulant le biais

Merci

Pepper Battleship

Détection de triche et confiance homme-machine

[À MODIFIER: Université - Master 1 Génie Logiciel]

Questions ?